

Model Checking

Model checking decision procedure

A model checking decision procedure, MCDP is an algorithm that checks if a formula ϕ is satisfied by an interpretation $\mathcal{I}$. Namely

$$\text{MCDP}(\phi, \mathcal{I}) = \text{true} \quad \text{if and only if} \quad \mathcal{I} \models \phi$$

$$\text{MCDP}(\phi, \mathcal{I}) = \text{false} \quad \text{if and only if} \quad \mathcal{I} \not\models \phi$$

Observation

The procedure of model checking returns for all inputs either **true** or **false** since for every interpretation $\mathcal{I}$ and for every formula ϕ , we have that either $\mathcal{I} \models \phi$ or $\mathcal{I} \not\models \phi$.

Observation

We have already seen a naive algorithm for model checking in propositional logic.

Satisfiability

Satisfiability decision procedure

A satisfiability decision procedure SDP is an algorithm that takes in input a formula ϕ and checks if ϕ is satisfiable. Namely

$$\begin{aligned}\text{SDP}(\phi) &= \textit{Satisfiable} && \text{if and only if } \mathcal{I} \models \phi \text{ for some } \mathcal{I} \\ \text{SDP}(\phi) &= \textit{Unsatisfiable} && \text{if and only if } \mathcal{I} \not\models \phi \text{ for all } \mathcal{I}\end{aligned}$$

When $\text{SDP}(\phi) = \textit{satisfiable}$, SDP can in addition return a model $\mathcal{I}$ (or, one of the models) of the formula ϕ .

Validity

Validity decision procedure

A decision procedure for Validity VDC, is an algorithm that checks whether a formula is valid. VDP can be based on a satisfiability decision procedure by exploiting the equivalence

ϕ is valid if and only if $\neg\phi$ is not satisfiable

$VDP(\phi) = true$ if and only if $SDP(\neg\phi) = Unsatisfiable$

$VDP(\phi) = false$ if and only if $SDP(\neg\phi) = Satisfiable$

When $SDP(\neg\phi)$ returns *Satisfiable*, it may in addition returns an interpretation $\mathcal{I}$, and $\mathcal{I}$ is called a **counter-model** for ϕ , in the sense that is an interpretation that falsifies ϕ .

Logical implication

Logical implication decision procedure

A decision procedure for logical consequence (or implication) LCDP is an algorithm that checks whether a formula ϕ is a logical consequence of a finite set of formulas $\Gamma = \{\gamma_1, \dots, \gamma_n\}$. LCDP can be implemented on the basis of satisfiability decision procedure by exploiting the property

$\Gamma \models \phi$ if and only if $\Gamma \cup \{\neg\phi\}$ is unsatisfiable

$LCDP(\Gamma, \phi) = \text{true}$ if and only if $SDP(\gamma_1 \wedge \dots \wedge \gamma_n \wedge \neg\phi) = \text{Unsatisfiable}$

$LCDP(\Gamma, \phi) = \text{false}$ if and only if $SDP(\gamma_1 \wedge \dots \wedge \gamma_n \wedge \neg\phi) = \text{Satisfiable}$

When $SDP(\gamma_1 \wedge \dots \wedge \gamma_n \wedge \neg\phi)$ returns *Satisfiable*, it may in addition return an interpretation $\mathcal{I}$, and $\mathcal{I}$ is a **model for Γ** and a **counter-model for ϕ** , i.e., a model of Γ that falsifies ϕ .

Proof of the previous property

Theorem

$\Gamma \models \phi$ if and only if $\Gamma \cup \{\neg\phi\}$ is unsatisfiable.

Proof.

- $\Rightarrow$ If $\Gamma \models \phi$, then every model of Γ satisfies ϕ , and therefore we cannot find a model of Γ where $\neg\phi$ is true. It follows that $\Gamma \cup \{\neg\phi\}$ is unsatisfiable.
- $\Leftarrow$ If $\Gamma \cup \{\neg\phi\}$ is unsatisfiable, then either (a) Γ itself is unsatisfiable, or (b) Γ has at least one model, but $\neg\phi$ is false in every model of Γ . If (a) is the case, then Γ logically implies everything, and therefore $\Gamma \models \phi$. If (b) is the case, the set of models of Γ is nonempty, and ϕ is true in all such model. So, we have shown that $\Gamma \models \phi$ holds in all cases.



Davis–Putnam–Logemann–Loveland (DPLL) procedure for satisfiability

DPLL SAT procedure:

- Introduced in 1961 by Martin Davis, George Logemann and Donald W. Loveland;
- Refinement of the earlier Davis–Putnam procedure, which is a resolution-based procedure developed by Davis and Hilary Putnam in 1960;
- After more than 60 years the DPLL procedure still forms the basis for most efficient complete SAT solvers.

Original references:

- Martin Davis, Hilary Putnam: A Computing Procedure for Quantification Theory. J. ACM 7(3): 201-215 (1960)
- Martin Davis, George Logemann, Donald W. Loveland: A machine program for theorem-proving. Commun. ACM 5(7): 394-397 (1962)

Conjunctive Normal form

Definition

- A **literal** is either a propositional variable or the negation of a propositional variable.

$$p, \neg q$$

- A **clause** is a disjunction of literals.

$$(a \vee \neg b \vee c)$$

- A formula is in **conjunctive normal form**, if it is a conjunction of clauses.

$$(p \vee \neg q \vee r) \wedge (q \vee r) \wedge (\neg p \vee \neg q) \wedge r$$

Conjunctive Normal form

Conjunctive Normal form

A formula in conjunctive normal form has the following shape:

$$(l_{11} \vee \cdots \vee l_{1n_1}) \wedge \cdots \wedge (l_{m1} \vee \cdots \vee l_{mn_m})$$

equivalently written as

$$\bigwedge_{i=1}^m \left(\bigvee_{j=1}^{n_j} l_{ij} \right)$$

where l_{ij} is the j -th literal of the i -th clause composing ϕ

Example

$$\begin{array}{ll} (p \vee \neg q) \wedge (r \vee p \vee \neg r) \wedge (p \vee p) & p \vee q \\ p \wedge q, & p \wedge \neg q \wedge (r \vee s) \end{array}$$

Equi-satisfiable formulas

Two formulas ϕ and ϕ' are **equi-satisfiable** iff:

ϕ is satisfiable if and only if ϕ' is satisfiable

- If two formulas are equi-satisfiable, are they equivalent? **No!**
- **Example:** Any satisfiable formula (e.g., p) is equi-satisfiable with $\top$, but clearly, $p \equiv \top$ is not valid!
- **Another example:** Introducing names leads to equi-satisfiable formulas. E.g. the formula $a \wedge b$ is equi-satisfiable of the formula $(n \equiv a \wedge b) \wedge n$, but it is not true that

$$(a \wedge b) \equiv (n \wedge (a \wedge b \equiv n))$$

- Equi-satisfiability is a weaker notion than equivalence. But useful if all we want to do is determine satisfiability.

Tseitin's transformation

Tseitin's transformation ...

... converts any propositional formula ϕ into an equi-satisfiable formula ϕ' in CNF with only a linear increase in size.

Key ideas:

- give a “name” to every subformula (except for literals), and use this name as a fresh propositional letter “representing” the subformula
- for every formula ψ of the form $p_1 \equiv (p_2 \circ p_3)$ (where $\circ$ is a connective), there is a CNF formula $CNF(\psi)$ that is equivalent to ψ
- transform the original formula ψ into a conjunction of many $CNF(\psi_i)$ obtained from ψ , one for each subformula of ψ

* Gregory S. Tseitin - *On the complexity of proof in propositional calculus*, Leningrad, 1968.

Tseitin's transformation

If ψ is any formula, then for every subformula ϕ of ψ , we define its name n_ϕ as follows:

- n_ϕ is simply ϕ if ϕ is a literal;
- n_ϕ is a new propositional letter if ϕ is not a literal.

The Tseitin's transformation $T(\psi)$ of ψ is the conjunction of

- n_ψ
- $CNF(q \equiv \neg n_\phi)$ for every non-literal subformula of the form $\neg\phi$ having name q
- $CNF(q \equiv (n_{\phi_1} \circ n_{\phi_2}))$ for every subformula of the form $\phi_1 \circ \phi_2$ having name q , where $\circ \in \{\vee, \wedge, \rightarrow, \equiv\}$.

Tseitin's transformation

We still need to understand what **CNF** does. We proceed to analyze all cases.

- **CNF** $(q \equiv \neg p) = (\neg q \vee \neg p) \wedge (p \vee q)$
- **CNF** $(q \equiv (p \wedge r)) = (q \vee \neg p \vee \neg r) \wedge (\neg q \vee p) \wedge (\neg q \vee r)$
- **CNF** $(q \equiv (p \vee r)) = (\neg q \vee p \vee r) \wedge (q \vee \neg p) \wedge (q \vee \neg r)$
- **CNF** $(q \equiv (p \rightarrow r)) = (\neg q \vee \neg p \vee r) \wedge (p \vee q) \wedge (\neg r \vee q)$
- **CNF** $(q \equiv (p \equiv r)) = (q \vee p \vee r) \wedge (q \vee \neg p \vee \neg r) \wedge (\neg q \vee p \vee \neg r) \wedge (\neg q \vee \neg p \vee r)$

Tseitin's transformation

Theorem

For every formula ψ , $T(\psi)$ is satisfiable if and only if ψ is. Moreover, $T(\psi)$ is 3-CNF formula whose size is linear with respect to the size of ψ .

Tseitin's transformation - Example

Let ψ be the formula $(p \vee q) \rightarrow (p \wedge \neg r)$

- 1 For each non-literal subformula, introduce new letters:

x_1 for ψ , x_2 for $p \vee q$, x_3 for $p \wedge \neg r$

- 2 Stipulate equivalences and convert them to CNF:

$$\text{CNF}(x_1 \equiv (x_2 \rightarrow x_3)) \Rightarrow \phi_1 : (\neg x_1 \vee \neg x_2 \vee x_3) \wedge (x_2 \vee x_1) \wedge (\neg x_3 \vee x_1)$$

$$\text{CNF}(x_2 \equiv (p \vee q)) \Rightarrow \phi_2 : (\neg x_2 \vee p \vee q) \wedge (\neg p \vee x_2) \wedge (\neg q \vee x_2)$$

$$\text{CNF}(x_3 \equiv (p \wedge \neg r)) \Rightarrow \phi_3 : (\neg x_3 \vee p) \wedge (\neg x_3 \vee \neg r) \wedge (\neg p \vee r \vee x_3)$$

- 3 The formula $T(\psi)$ is:

$$x_1 \wedge \phi_1 \wedge \phi_2 \wedge \phi_3$$

Properties of $\wedge$ and $\vee$

Commutativity of $\wedge$: $\phi \wedge \psi \equiv \psi \wedge \phi$

Commutativity of $\vee$: $\phi \vee \psi \equiv \psi \vee \phi$

Absorption of $\wedge$: $\phi \wedge \phi \equiv \phi$

Absorption of $\vee$: $\phi \vee \phi \equiv \phi$